

Teoria da Computação

Autômatos Finitos e Linguagens Regulares

Prof. Renê R. Veloso

Operações Regulares

DEFINIÇÃO 1.23

Sejam A e B linguagens. Definimos as operações regulares *união*, *concatenação*, e *estrela* da seguinte forma.

- **União:** $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$.
- **Concatenação:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.
- **Estrela:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e cada } x_i \in A\}$.

EXEMPLO 1.24

Suponha que o alfabeto Σ seja o alfabeto padrão de 26 letras $\{a, b, \dots, z\}$. Se $A = \{\text{legal}, \text{ruim}\}$ e $B = \{\text{garoto}, \text{garota}\}$, então

$$A \cup B = \{\text{legal}, \text{ruim}, \text{garoto}, \text{garota}\},$$

$$A \circ B = \{\text{legalgaroto}, \text{legalgarota}, \text{ruimgaroto}, \text{ruimgarota}\}, \text{ e}$$

$$A^* = \{\epsilon, \text{legal}, \text{ruim}, \text{legallegal}, \text{legalruim}, \text{ruimlegal}, \text{ruimruim}, \\ \text{legallegallegal}, \text{legallegalruim}, \text{legalruimlegal}, \\ \text{legalruimruim}, \dots\}.$$

Operações Regulares

DEFINIÇÃO 1.23

Sejam A e B linguagens. Definimos as operações regulares *união*, *concatenação*, e *estrela* da seguinte forma.

- **União:** $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$.
- **Concatenação:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$.
- **Estrela:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e cada } x_i \in A\}$.

Fechamento:

Seja $\mathcal{N} = \{1, 2, 3, \dots\}$ o conjunto dos números naturais. Quando dizemos que $\mathcal{N}$ é *fechado sob multiplicação* queremos dizer que, para quaisquer x e y em $\mathcal{N}$, o produto $x \times y$ também está em $\mathcal{N}$. Diferentemente, $\mathcal{N}$ não é fechado sob divisão, pois 1 e 2 estão em $\mathcal{N}$ mas $1/2$ não está. Em geral, uma coleção de objetos é *fechada* sob alguma operação se, aplicando-se essa operação a membros da coleção, recebe-se um objeto ainda na coleção. Mostramos que a coleção de linguagens regulares é fechada sob todas as três das operações regulares. Na Seção 1.3 mostramos que essas são ferramentas úteis para se manipular linguagens regulares e entender o poder de autômatos finitos. Começamos com a operação de união.

Operações Regulares: União

TEOREMA 1.25

A classe de linguagens regulares é fechada sob a operação de união.

Em outras palavras, se A_1 e A_2 são linguagens regulares, o mesmo acontece com $A_1 \cup A_2$.

PROVA

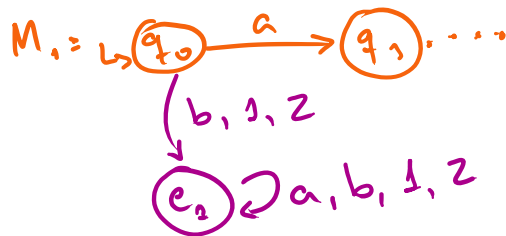
Suponha que M_1 reconheça A_1 , onde $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, e que M_2 reconheça A_2 , onde $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

δ_1 e δ_2 devem ser totais!
(adicionar estados de erro aos AFDs)
considerando $\Sigma = \Sigma_1 \cup \Sigma_2$

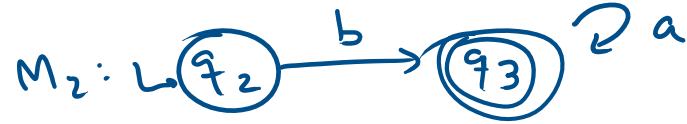
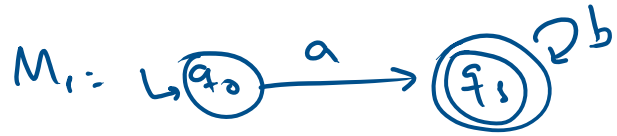
Construa M para reconhecer $A_1 \cup A_2$, onde $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\}$. $Q_1 \times Q_2$
2. Σ , o alfabeto, é o mesmo em M_1 e M_2 . $\Sigma = \Sigma_1 \cup \Sigma_2$.
3. $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.
4. q_0 é o par (q_1, q_2) .
5. $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

Exemplo: $\Sigma_1 = \{a, b\}$ e $\Sigma_2 = \{1, 2\}$

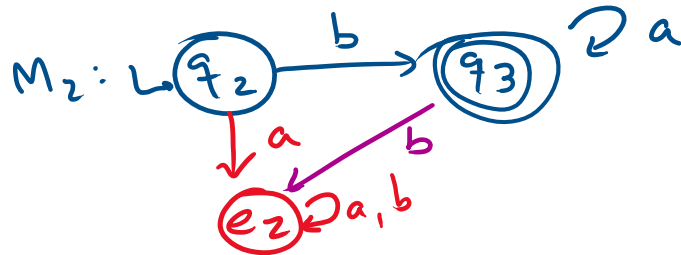
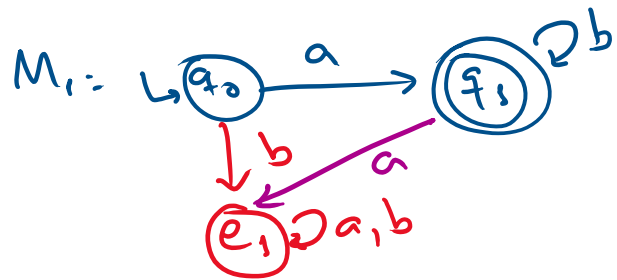


Operações Regulares: União

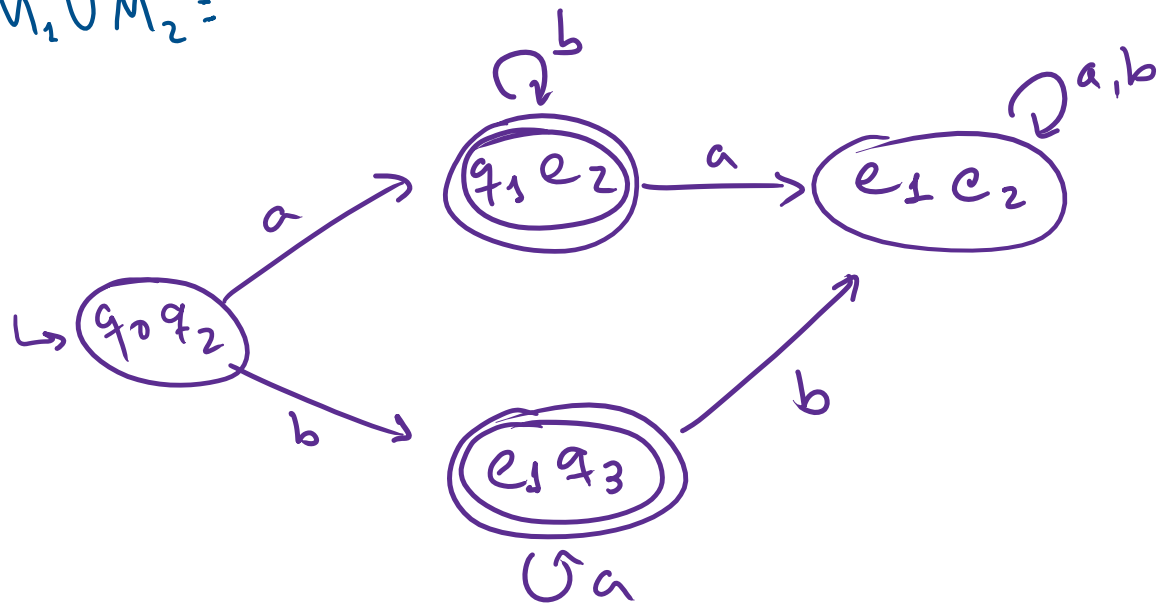


$M_1 \cup M_2 =$

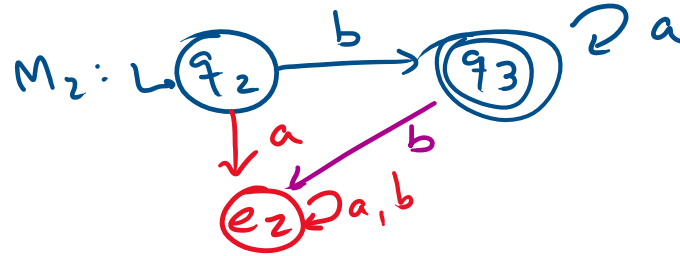
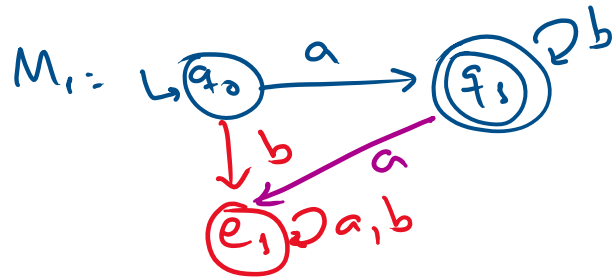
Operações Regulares: União



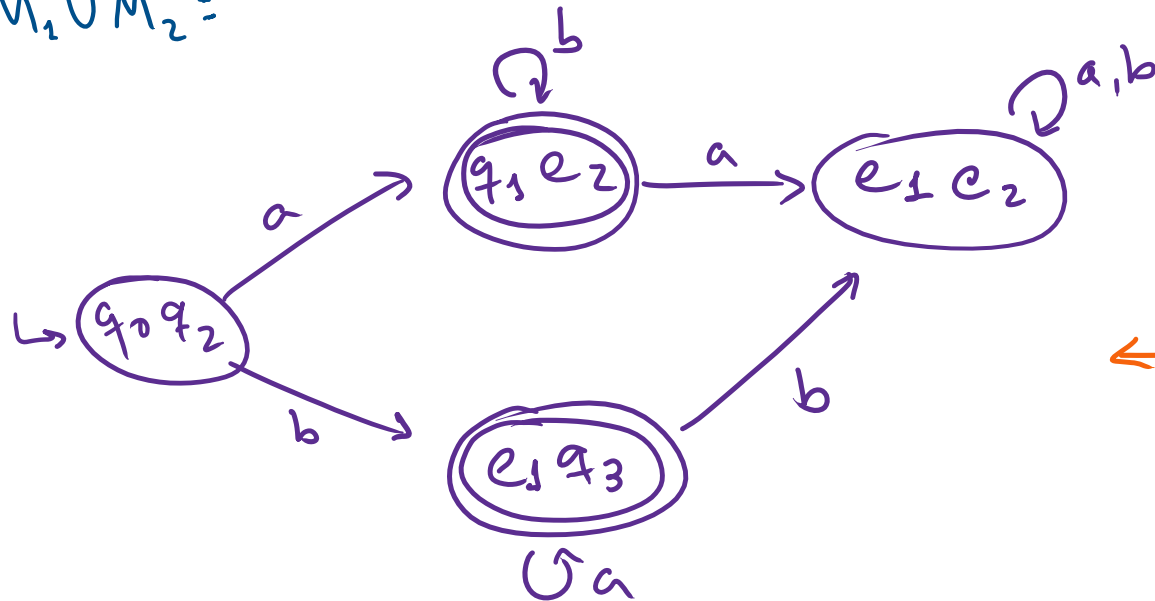
$M_1 \cup M_2$:



Operações Regulares: União



$M_1 \cup M_2$:



$Q_1 \times Q_2 =$

$\{ (q_0, q_2), (q_0, q_3), (q_0, e_2), (q_1, q_2), (q_1, q_3), \dots \}$

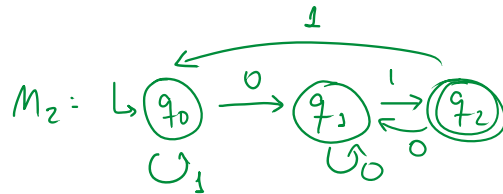
← apenas alguns pares usados

Operações Regulares: União

$L_1 = \{w \in \{0,1\}^* : w \text{ possui um número par de 0's}\}$

$L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01\}$

Pede-se: encontrar um AFD para reconhecer $L_1 \cup L_2 = \{w \in \{0,1\}^* : w \text{ termina em } 01 \text{ ou possui n}^\circ \text{ par de 0's}\}$



Operações Regulares: Concatenação

TEOREMA 1.26

A classe de linguagens regulares é fechada sob a operação de concatenação.

Em outras palavras, se A_1 e A_2 são linguagens regulares então o mesmo acontece com $A_1 \circ A_2$.

$$A \circ B = \{xy : x \in A \text{ e } y \in B\}.$$

* $A \circ B$ pode ser escrito AB

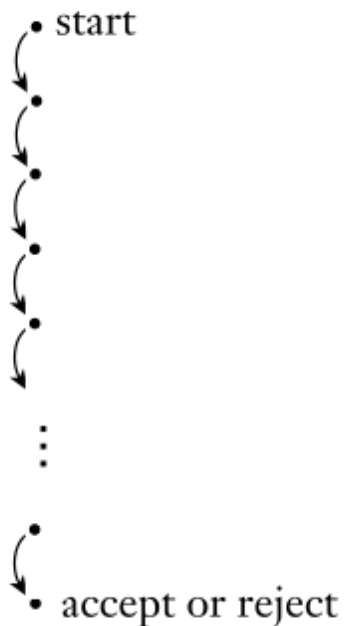
... em vez de construir o autômato M para aceitar sua entrada se M_1 ou M_2 aceitam, ele tem que aceitar se sua entrada puder ser quebrada em duas partes, onde M_1 aceita a primeira parte e M_2 aceita a segunda parte.

Exemplo: $A = \{x : x \in \alpha a, \alpha \in \{a, b\}^*\}$
 $B = \{y : y \in \alpha a, \alpha \in \{a, b\}^*\}$
 $AB = \{aa, baacab, baab, \dots\}$

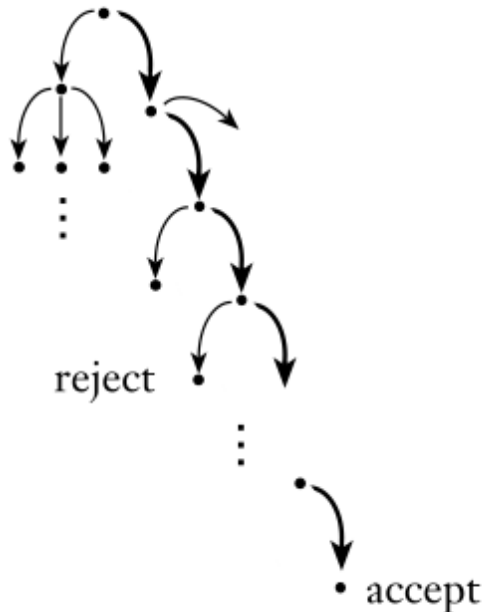
Qual é o problema?

Não-determinismo

Deterministic
computation



Nondeterministic
computation



Não-determinismo pode ser visto como uma espécie de computação paralela na qual múltiplos e independentes “processos” ou “threads” podem estar rodando concorrentemente. Quando o AFN se divide para seguir as diversas escolhas, isso corresponde a um processo “bifurcar” em vários filhos, cada um procedendo separadamente.

Se pelo menos um desses processos aceita, então a computação inteira aceita.

AFD: Autômato Finito Determinístico

AFN: Autômato Finito Não-determinístico

A.F. Não-determinístico

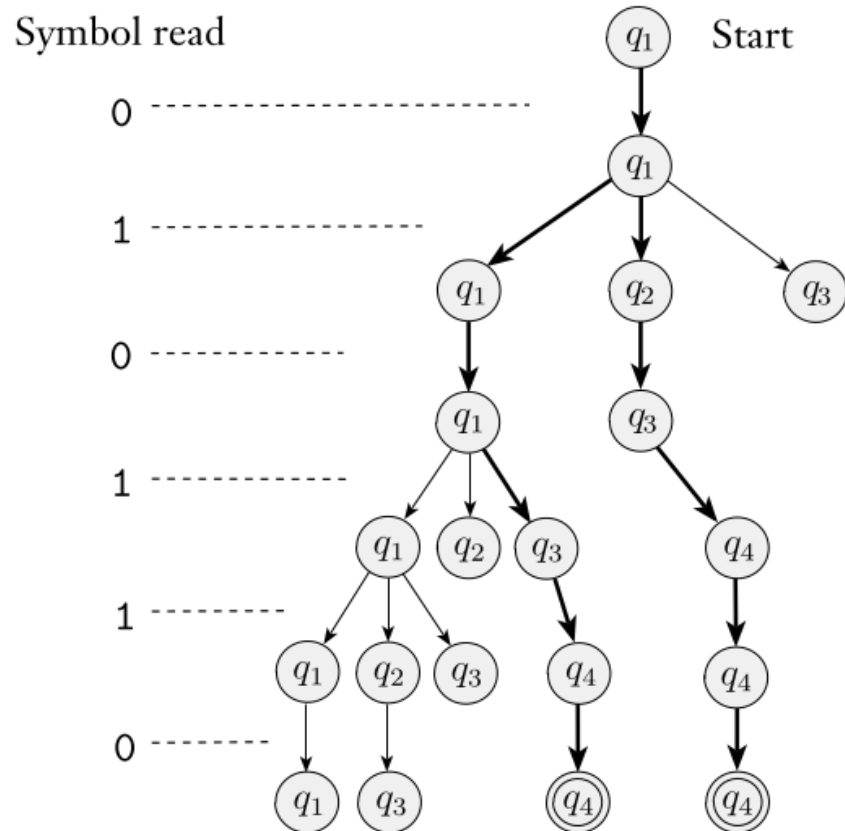
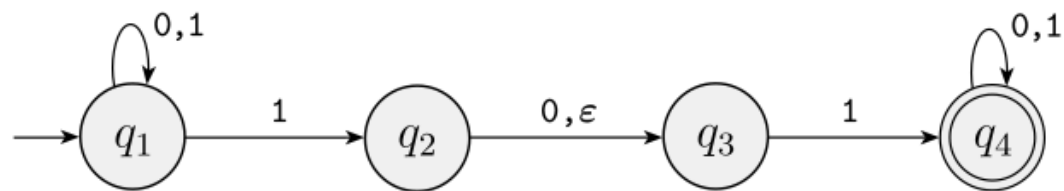


FIGURA 1.29

A computação de N_1 sobre a entrada 010110

A.F. Não-determinístico

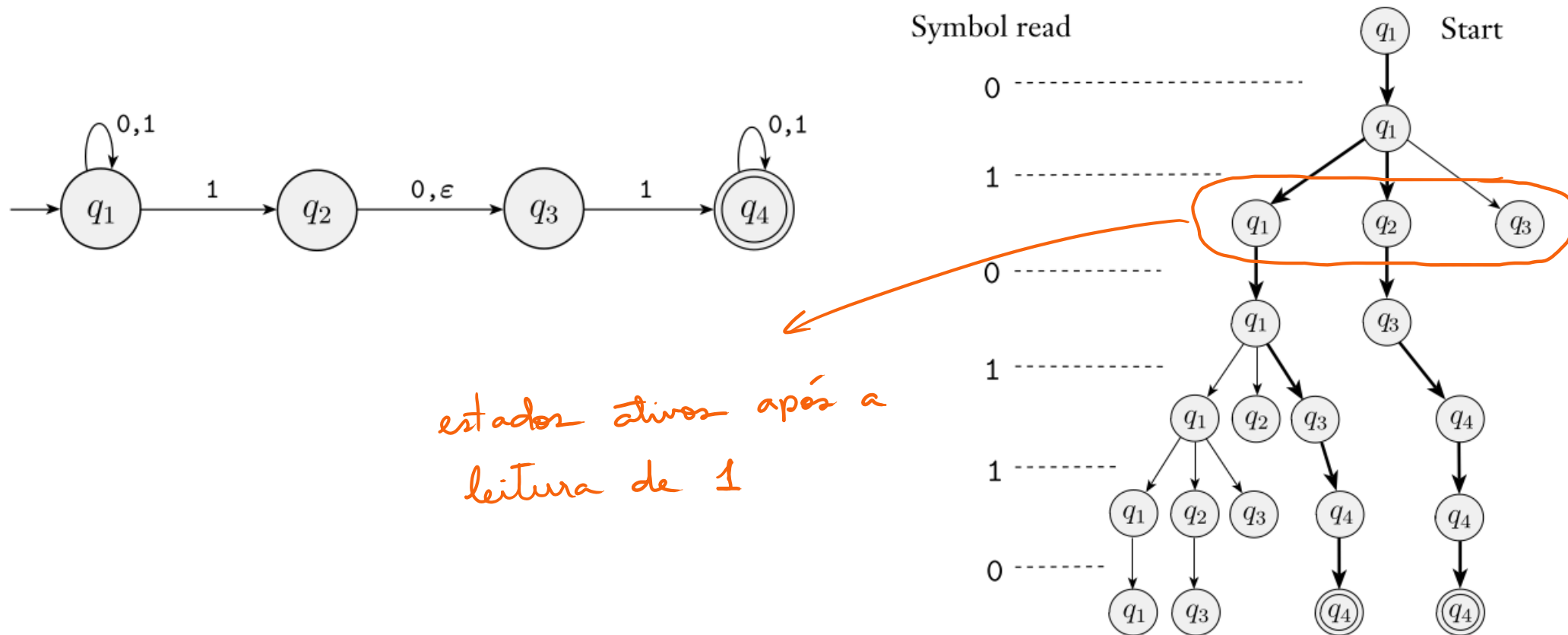
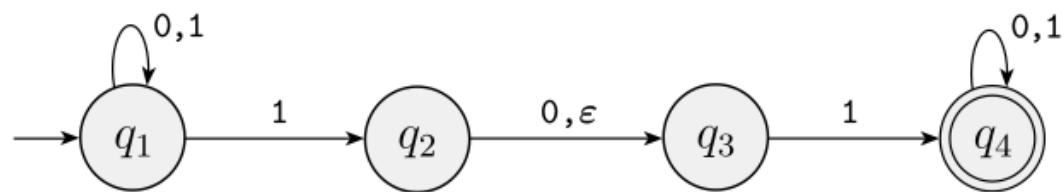


FIGURA 1.29

A computação de N_1 sobre a entrada 010110

A.F. Não-determinístico



*estados ativos após a
leitura de 0*

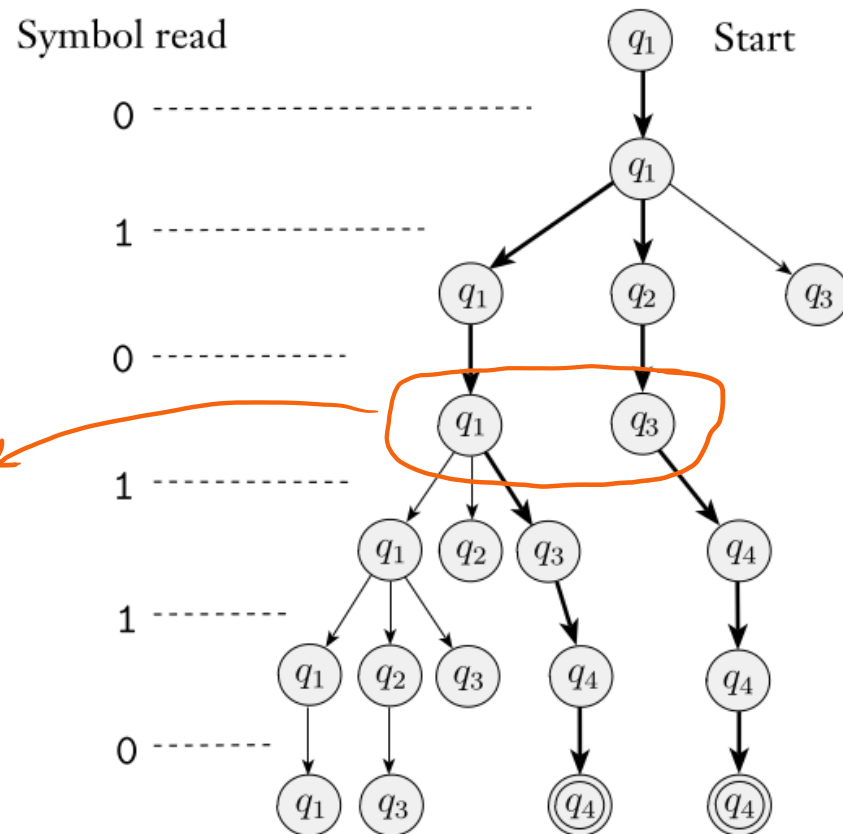
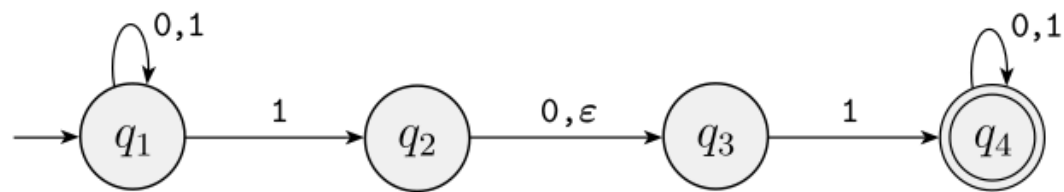


FIGURA 1.29

A computação de N_1 sobre a entrada 010110

A.F. Não-determinístico



*estados ativos após a
leitura de 1*

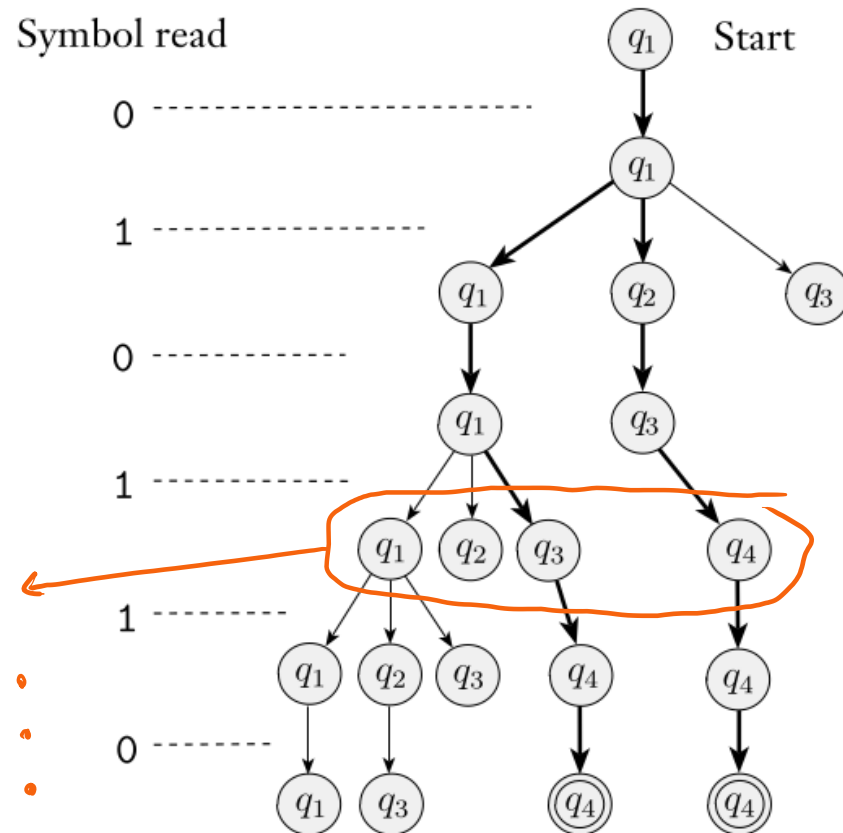


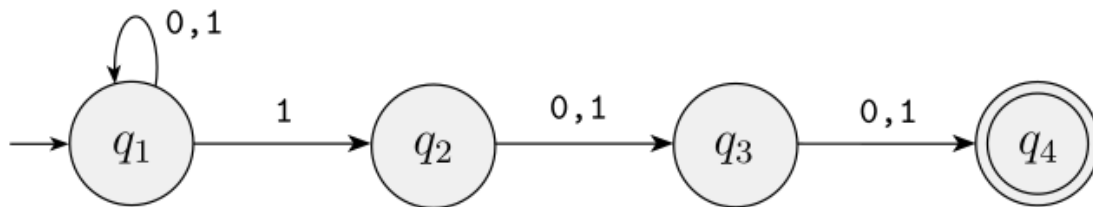
FIGURA 1.29

A computação de N_1 sobre a entrada 010110

A.F. Não-determinístico (oráculo)

EXEMPLO 1.30

Seja A a linguagem consistindo de todas as cadeias sobre $\{0,1\}$ contendo um 1 na terceira posição a partir do final (e.g., 000100 está em A mas 0011 não). O seguinte AFN de quatro estados N_2 reconhece A .

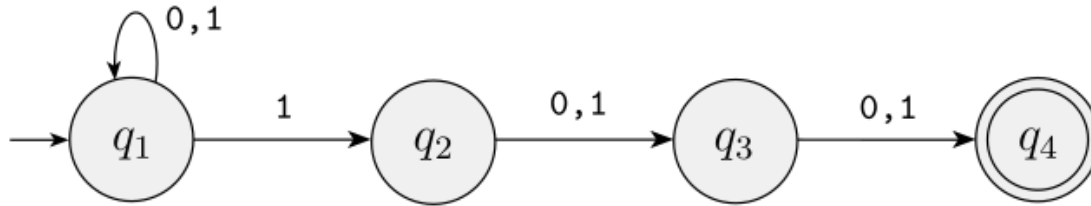


Uma boa maneira de ver a computação desse AFN é dizer que ele permanece no estado inicial q_1 até que ele “adivinhe” que ele está a três posições do final. Nesse ponto, se o símbolo de entrada for um 1, ele ramifica para o estado q_2 e usa q_3 e q_4 para “chechar” se sua adivinhação estava correta.

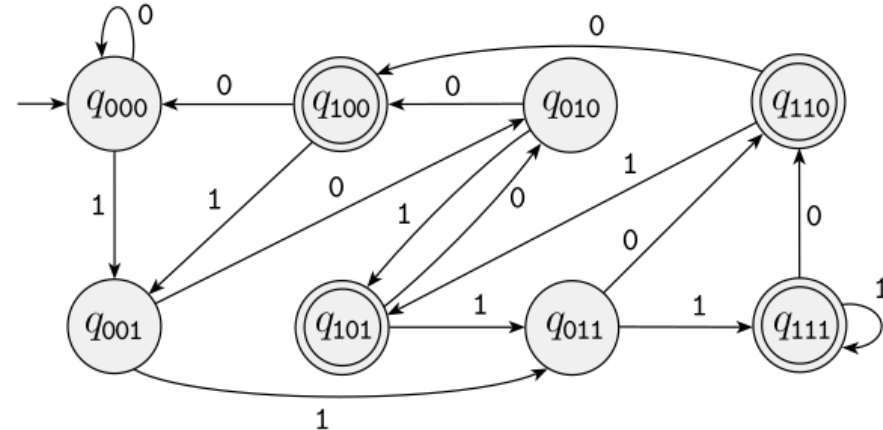
A.F. Não-determinístico (oráculo)

EXEMPLO 1.30

Seja A a linguagem consistindo de todas as cadeias sobre $\{0,1\}$ contendo um 1 na terceira posição a partir do final (e.g., 000100 está em A mas 0011 não). O seguinte AFN de quatro estados N_2 reconhece A .



AFD correspondente



A.F. Não-determinístico

DEFINIÇÃO 1.37

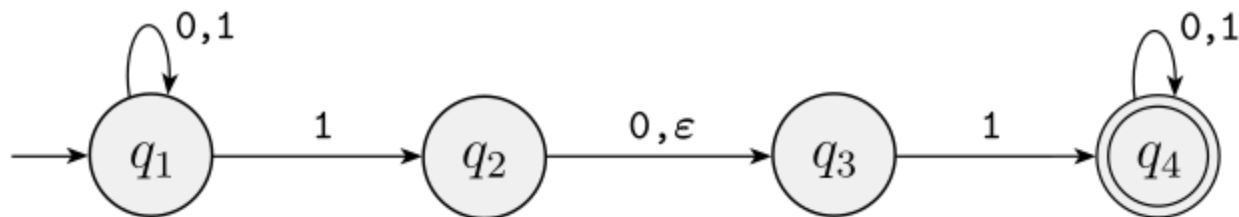
Um *autômato finito não-determinístico* é uma 5-upla $(Q, \Sigma, \delta, q_0, F)$, onde

1. Q é um conjunto finito de estados,
2. Σ é um alfabeto finito,
3. $\delta: Q \times \Sigma_\epsilon \longrightarrow \mathcal{P}(Q)$ é a função de transição,
4. $q_0 \in Q$ é o estado inicial, e
5. $F \subseteq Q$ é o conjunto de estados de aceitação.

Sendas: $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$

Seja $B = \{a, b\}$, $2^B = P(B) = \{\phi, \{a\}, \{b\}, \{a, b\}\}$

A.F. Não-determinístico



A descrição formal de N_1 é $(Q, \Sigma, \delta, q_1, F)$, onde

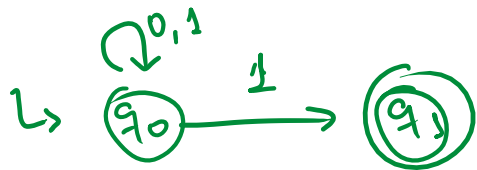
1. $Q = \{q_1, q_2, q_3, q_4\}$,
2. $\Sigma = \{0,1\}$,
3. δ é dado como

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 é o estado inicial, e
5. $F = \{q_4\}$.

A.F. Não-determinístico

Exemplo: $L = \{w \in \{0,1\}^* : \underline{w \text{ termina em } 1}\}$
formato $\alpha 1$, com $\alpha \in \{0,1\}^*$



$$N_L = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

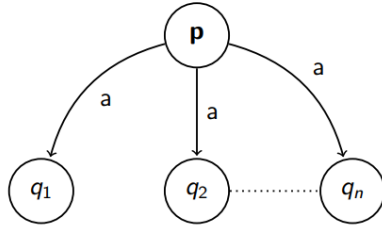
δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\emptyset$

... mas e o AFD ?

... de AFN para AFD

TEOREMA 1.39

Todo autômato finito não-determinístico tem um autômato finito determinístico equivalente.



Se k for o número de estados num AFN, este AFN tem 2^k subconjuntos de estados possíveis.

Cada subconjunto corresponde a uma das possibilidades de transição do AFD, portanto o AFD simulado terá 2^k estados.

... de AFN para AFD

PROVA Seja $N = (Q, \Sigma, \delta, q_0, F)$ o AFN que reconhece alguma linguagem A . Construímos um AFD $M = (Q', \Sigma, \delta', q_0', F')$ que reconhece A . Antes de realizar a construção completa, vamos primeiro considerar o caso mais fácil no qual N não tem setas ϵ . Mais adiante levamos as setas ϵ em consideração.

1. $Q' = \mathcal{P}(Q)$.

Todo estado de M é um conjunto de estados de N . Lembre-se de que $\mathcal{P}(Q)$ é o conjunto de subconjuntos de Q .

2. Para $R \in Q'$ e $a \in \Sigma$ let $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ para algum } r \in R\}$. Se R é um estado de M , ele também é um conjunto de estados de N . Quando M lê um símbolo a no estado R , ele mostra para onde a leva cada estado em R . Dado que cada estado pode ir para um conjunto de estados, tomamos a união de todos esses conjuntos. Uma outra maneira de escrever essa expressão é

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).^4$$

3. $q_0' = \{q_0\}$.

M começa no estado correspondente à coleção contendo somente o estado inicial de N .

4. $F' = \{R \in Q' \mid R \text{ contém um estado de aceitação de } N\}$.

A máquina M aceita se um dos possíveis estados no quais N poderia estar nesse ponto é um estado de aceitação.

PROVA Seja $N = (Q, \Sigma, \delta, q_0, F)$ o AFN que reconhece alguma linguagem A . Construímos um AFD $M = (Q', \Sigma, \delta', q_0', F')$ que reconhece A . Antes de realizar a construção completa, vamos primeiro considerar o caso mais fácil no qual N não tem setas ϵ . Mais adiante levamos as setas ϵ em consideração.

1. $Q' = \mathcal{P}(Q)$.

Todo estado de M é um conjunto de estados de N . Lembre-se de que $\mathcal{P}(Q)$ é o conjunto de subconjuntos de Q .

2. Para $R \in Q'$ e $a \in \Sigma$ let $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ para algum } r \in R\}$. Se R é um estado de M , ele também é um conjunto de estados de N . Quando M lê um símbolo a no estado R , ele mostra para onde a leva cada estado em R . Dado que cada estado pode ir para um conjunto de estados, tomamos a união de todos esses conjuntos. Uma outra maneira de escrever essa expressão é

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).^4$$

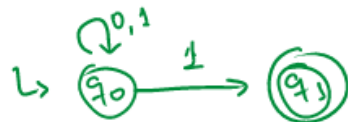
3. $q_0' = \{q_0\}$.

M começa no estado correspondente à coleção contendo somente o estado inicial de N .

4. $F' = \{R \in Q' \mid R \text{ contém um estado de aceitação de } N\}$.

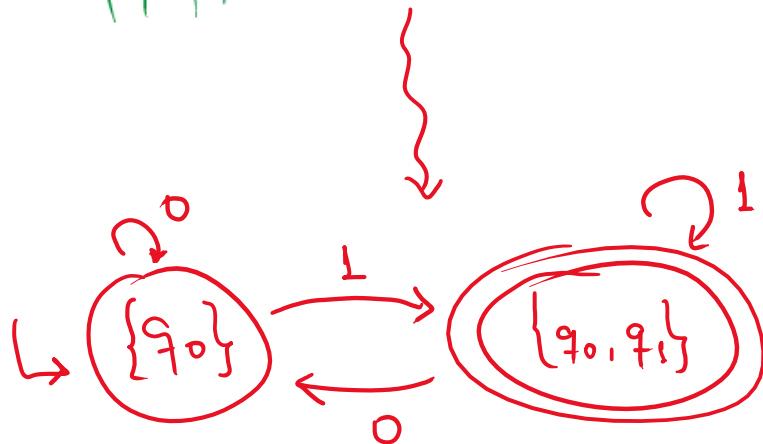
A máquina M aceita se um dos possíveis estados no quais N poderia estar nesse ponto é um estado de aceitação.

... de AFN para AFD



$$N = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

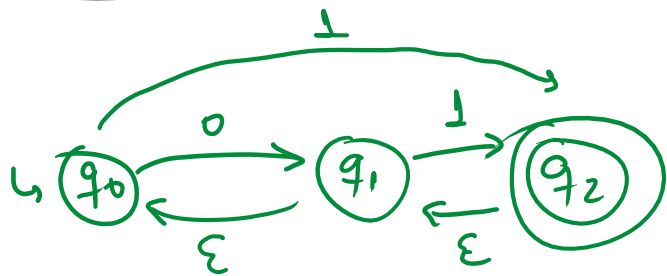
δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\emptyset$



A.F. Não-determinístico

Exemplo: $L = \{w \in \{0,1\}^* : \underline{w \text{ termina em } 1}\}$

formato $\alpha 1$, com $\alpha \in \{0,1\}^*$



$N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

δ	0	1	ϵ
q_0	$\{q_1\}$	$\{q_2\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\{q_0\}$
q_2	$\emptyset$	$\emptyset$	$\{q_1\}$

... mas e o AFD ?

PROVA Seja $N = (Q, \Sigma, \delta, q_0, F)$ o AFN que reconhece alguma linguagem A . Construímos um AFD $M = (Q', \Sigma, \delta', q_0', F')$ que reconhece A . Antes de realizar a construção completa, vamos primeiro considerar o caso mais fácil no qual N não tem setas ϵ . Mais adiante levamos as setas ϵ em consideração.

1. $Q' = \mathcal{P}(Q)$.

Todo estado de M é um conjunto de estados de N . Lembre-se de que $\mathcal{P}(Q)$ é o conjunto de subconjuntos de Q .

2. Para $R \in Q'$ e $a \in \Sigma$ let $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ para algum } r \in R\}$. Se R é um estado de M , ele também é um conjunto de estados de N . Quando M lê um símbolo a no estado R , ele mostra para onde a leva cada estado em R . Dado que cada estado pode ir para um conjunto de estados, tomamos a união de todos esses conjuntos. Uma outra maneira de escrever essa expressão é

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

3. $q_0' = \{q_0\}$.

M começa no estado correspondente à coleção contendo somente o estado inicial de N .

4. $F' = \{R \in Q' \mid R \text{ contém um estado de aceitação de } N\}$.

A máquina M aceita se um dos possíveis estados no quais N poderia estar nesse ponto é um estado de aceitação.

Agora precisamos considerar as setas ϵ . Para fazer isso fixamos um pouco mais de notação. Para qualquer estado R de M definimos $E(R)$ como sendo a coleção de estados que podem ser atingidos a partir de R indo somente ao longo de setas ϵ , incluindo os próprios membros de R . Formalmente, para $R \subseteq Q$ seja

$$E(R) = \{q \mid q \text{ pode ser atingido a partir de } R \text{ viajando-se ao longo de 0 ou mais setas } \epsilon\}.$$

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ para algum } r \in R\}.$$

Mudando q_0' para $E(\{q_0\})$

atualização para transições ϵ

ϵ -fechamento ou fecho-vazio

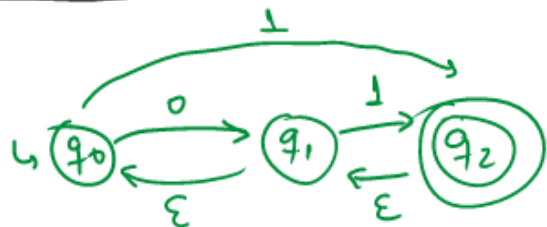
$$E(R) = \bigcup_{r \in R} E(r)$$

... de AFN para AFD

Fecho-vazio (E)

Exemplo: $L = \{w \in \{0,1\}^* : w \text{ termina em } 1\}$

formato $\alpha 1$, com $\alpha \in \{0,1\}^*$



$E(q_0) =$ quem fica ativo quando q_0 está ativo?
 $= \{q_0\}$

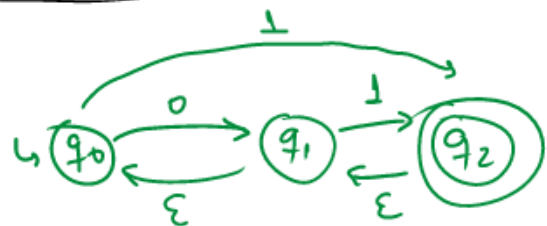
$E(q_1) = \{q_0, q_1\}$

$E(q_2) = \{q_0, q_1, q_2\}$

A.F. Não-determinístico

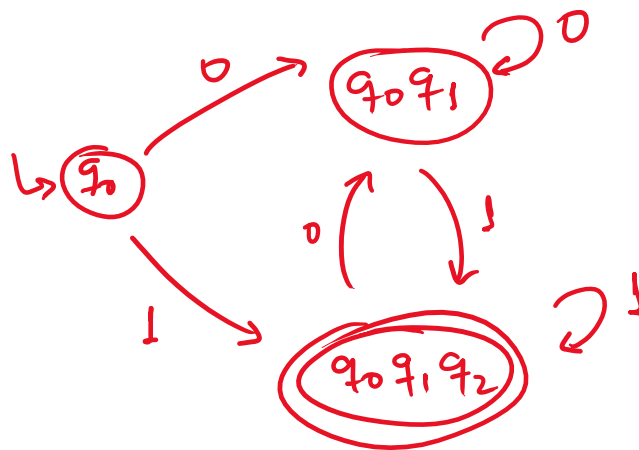
Exemplo: $L = \{w \in \{0,1\}^* : w \text{ termina em } 1\}$

formato $\alpha 1$, com $\alpha \in \{0,1\}^*$



$N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

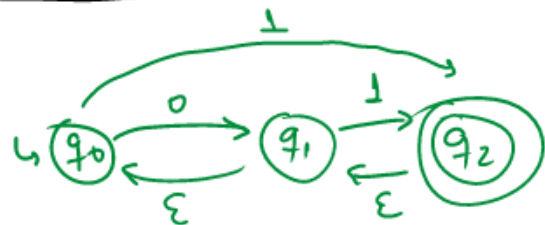
δ	0	1	ϵ
q_0	$\{q_1\}$	$\{q_2\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\{q_0\}$
q_2	$\emptyset$	$\emptyset$	$\{q_1\}$



A.F. Não-determinístico

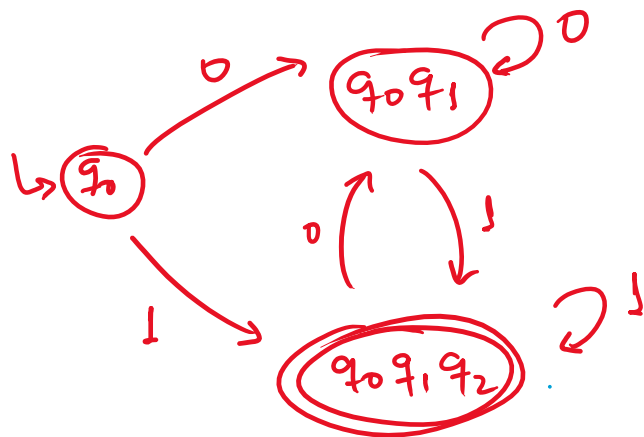
Exemplo: $L = \{w \in \{0,1\}^* : w \text{ termina em } 1\}$

formato $\alpha 1$, com $\alpha \in \{0,1\}^*$

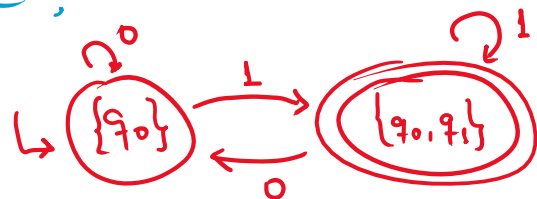


$N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$

δ	0	1	ϵ
q_0	$\{q_1\}$	$\{q_2\}$	$\emptyset$
q_1	$\emptyset$	$\{q_2\}$	$\{q_0\}$
q_2	$\emptyset$	$\emptyset$	$\{q_1\}$

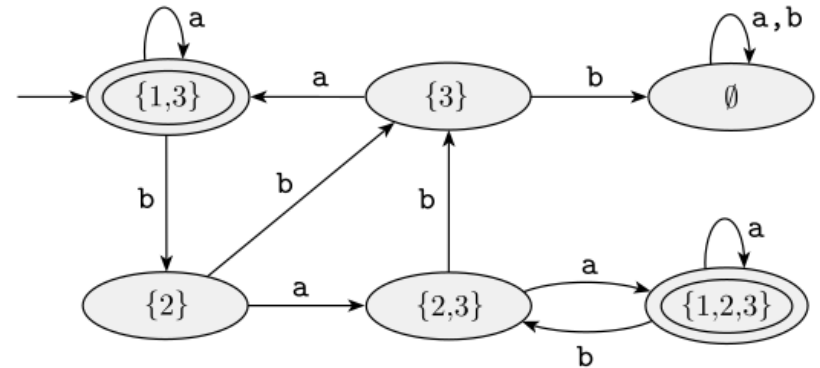
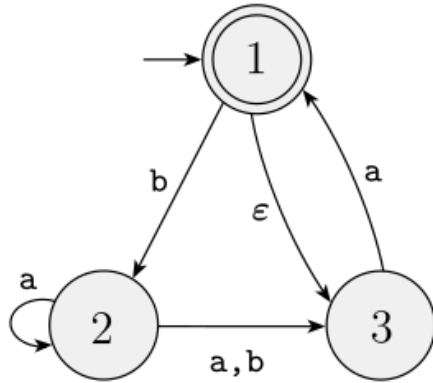


diferente!



... de AFN para AFD (exemplo 1.41)

$\{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$.



Exercícios

Fazer os Exercícios 1.7 e 1.8

Próxima aula:

- **Continuação: operações regulares**
- função de transição estendida para AFNs
- Aceite em um AFN (computação)